PROGRAMMING & CS FUNDAMENTALS

Concurrency vs Parallelism Visual Guide

Doing many things at once —
or just appearing to? The distinction that decides design.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Concurrency != Parallelism

The single most confused pair in systems programming.

Concurrency

dealing with many■things at once

Parallelism

DOING many things■at once

One barista juggling 4 orders = concurrent. Four baristas = parallel.

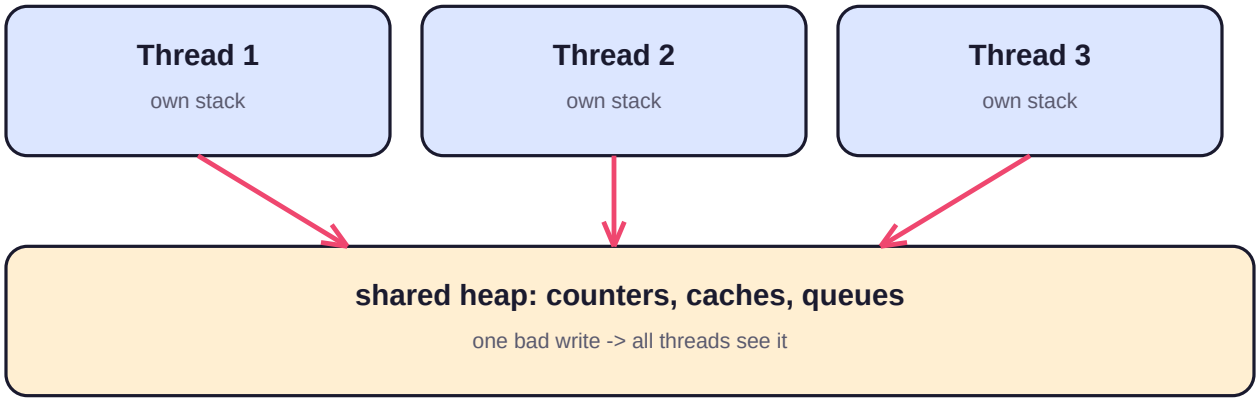
Concurrency is about STRUCTURE (interleaving); parallelism is about EXECUTION (simultaneous).

- You can have concurrency WITHOUT parallelism: one core interleaving tasks.
- Parallelism implies concurrency, not vice versa.
- Async I/O is concurrency; multi-core number crunching is parallelism.

STEP 2/6

Threads: Shared-Memory Concurrency

Multiple flows, one address space — fast to share, easy to corrupt.



- Threads share memory: communication is cheap (a pointer), coordination is hard (locks).
- Preemptive scheduling: the OS can pause you ANYWHERE — every shared access is a race candidate.
- Use for: I/O-bound work, responsive UIs, servers handling many connections.

STEP 3/6

Async: One Thread, Many Waiters

Cooperative multitasking — tasks yield instead of being preempted.

```
async def fetch(url):
    resp = await http.get(url)   # YIELD: run others while waiting
    return resp.json()

await asyncio.gather(*(fetch(u) for u in urls))  # 1000s of tasks, 1 thread
```

event loop

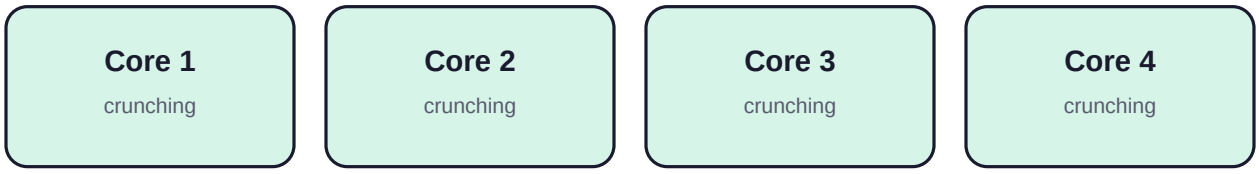
tasks yield at await — no locks between yields

- No preemption BETWEEN awaits: shared state is safe until you yield — fewer races by construction.
- One blocking call (time.sleep, sync I/O) freezes EVERYTHING — never block the loop.
- Scales to 10k+ connections where threads would drown in stack memory.

STEP 4/6

Parallelism: True Simultaneity

Multiple cores, real speedup — for CPU-bound work.




```
# threads won't help CPU-bound Python (GIL!) — use processes
with ProcessPoolExecutor() as ex:
    results = ex.map(train_model, shards)  # one process per core
```

- CPU-bound + threads + GIL (Python/Ruby) = no speedup — processes or native code instead.
- Data parallelism: split the DATA (map-reduce, SIMD); task parallelism: split the WORK.
- Overhead matters: spawning a process per tiny task loses — batch the work.

STEP 5/6

Amdahl's Law: The Speedup Ceiling

The sequential fraction caps EVERYTHING.

```
speedup(n) = 1 / (s + (1 - s) / n)
# s = sequential fraction, n = cores

s = 10%, n = 100 cores  ->  speedup ~ 9.2x   (NOT 100x!)
s = 10%, n = infinity   ->  speedup = 10x max
```

Halve the sequential part and you double the ceiling — optimize s before buying cores.

- Gustafson's counter: bigger problems parallelize better — scale the WORK, not just cores.
- Diminishing returns hit fast: 8 -> 16 cores rarely doubles real throughput.
- Measure s by profiling — it hides in startup, I/O, and lock contention.

STEP 6/6

Model Chooser

Match the model to the bottleneck.

async/await

I/O-bound, many connections — 10k conns, one thread

threads

I/O-bound, blocking libs — simple, OS-scheduled

processes / threads*

CPU-bound, multicore — true parallelism

multiprocessing

CPU-bound, Python — escape the GIL

async + thread pool

Mixed pipeline — loop for I/O, pool for CPU bursts

*threads for CPU-bound only where no GIL (Java, Go, Rust) or work is in native code (numpy).

STEP ✓I✓

Key Takeaways

Concurrency vs Parallelism — Remember

- Concurrency = dealing with many things (interleaving); parallelism = doing many things (simultaneous).
- Threads share memory: cheap communication, hard coordination — every shared access can race.
- Async: cooperative, one thread, yield at await — massive I/O scale, never block the loop.
- CPU-bound work needs true parallelism: processes (or GIL-free threads); mind Amdahl's ceiling.
- Amdahl: speedup limited by the sequential fraction — 10% serial caps you near 10x forever.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev